

Database Management Systems (DBMS)

Solved Model Question Paper (Feb/March 2025)

Subject: Database Management Systems

Term: Feb/March SEE 2025

Topic: Optimization, Normalization, Transactions, and NoSQL

Part A: Analytical & Logical Reasoning

Q1. Functional Dependency Proof

Question: Consider the set of functional dependencies

$F = \{A \rightarrow BC, CD \rightarrow EF, C \rightarrow E\}$. Show that $AD \rightarrow F$ holds in F .

Solution: To prove $AD \rightarrow F$, we must calculate the attribute closure of $\{A, D\}$ denoted as $(AD)^+$.

1. **Start:** Initial closure = $\{A, D\}$
2. **Apply $A \rightarrow BC$:** Since A is present, add B and C .
 - Closure = $\{A, D, B, C\}$
3. **Apply $C \rightarrow E$:** Since C is present, add E .
 - Closure = $\{A, B, C, D, E\}$
4. **Apply $CD \rightarrow EF$:** We have both C and D in the closure. Therefore, we can determine E and F . (E is already there, add F).
 - Closure = $\{A, B, C, D, E, F\}$

Conclusion: Since F is in $(AD)^+$, the dependency $AD \rightarrow F$ holds.

Q2. Decomposition Analysis

Question: Consider a schema $R(A, B, C, D)$ and functional dependencies $A \rightarrow B$ and $C \rightarrow D$. Determine whether the decomposition of R into $R_1(A, B)$ and $R_2(C, D)$ has the **dependency preservation property** and **lossless join property**.

Solution:

1. Dependency Preservation:

- $FD_1 : A \rightarrow B$ contains attributes $\{A, B\}$, which are all present in relation $R_1(A, B)$.
Preserved.
- $FD_2 : C \rightarrow D$ contains attributes $\{C, D\}$, which are all present in relation $R_2(C, D)$.
Preserved.
- **Result:** The decomposition is **Dependency Preserving**.

2. Lossless Join Property: For a decomposition to be lossless, the intersection of the decomposed relations must contain a key for at least one of the sub-relations.

- $R_1 \cap R_2 = \{A, B\} \cap \{C, D\} = \emptyset$ (Empty Set)
- Since there are no common attributes to join on, the natural join $R_1 \bowtie R_2$ will result in a Cartesian Product, creating spurious tuples.
- **Result:** The decomposition is **Lossy** (Not Lossless).

Q3. Instance Validation

Question: Given the following relation instance of $R(W, X, Y, Z)$:

W	X	Y	Z
1	1	4	2
2	1	5	3
3	1	6	3
4	1	6	4

Which of the following functional dependencies are satisfied? a) $WX \rightarrow Y$

b) $Y \rightarrow Z$

c) $XZ \rightarrow W$

d) $Z \rightarrow X$

Solution:

- **a) $WX \rightarrow Y$ (Satisfied )**
 - The combination (W, X) is unique for every row: $(1, 1), (2, 1), (3, 1), (4, 1)$. Since there are no duplicate Left-Hand Side (LHS) values, the dependency cannot be violated.
- **b) $Y \rightarrow Z$ (Violated )**
 - Look at rows where $Y = 6$ (Rows 3 and 4).
 - In Row 3: $Y = 6 \rightarrow Z = 3$.
 - In Row 4: $Y = 6 \rightarrow Z = 4$.
 - Since $3 \neq 4$, the dependency does not hold.
- **c) $XZ \rightarrow W$ (Violated )**
 - Look at rows where $XZ = (1, 3)$ (Rows 2 and 3).
 - In Row 2: $(X = 1, Z = 3) \rightarrow W = 2$.
 - In Row 3: $(X = 1, Z = 3) \rightarrow W = 3$.

- Since $2 \neq 3$, the dependency does not hold.
- **d) $Z \rightarrow X$ (Satisfied )**
 - Look at duplicates in Z : $Z = 3$ appears in Rows 2 and 3.
 - In Row 2: $Z = 3 \rightarrow X = 1$.
 - In Row 3: $Z = 3 \rightarrow X = 1$.
 - For all other values ($Z = 2, Z = 4$), X is consistently 1. The dependency holds.

Part B: Conceptual Differences & Definitions

Q4. Relation vs. Relation Schema

Question: Write the difference between a relation and a relation schema.

Feature	Relation Schema	Relation (Instance)
Definition	The logical design or blueprint of the database.	The set of data (tuples) stored in the database at a specific moment in time.
Stability	Static; changes very rarely (e.g., adding a column).	Dynamic; changes frequently (insert, update, delete).
Example	Student(ID, Name, Age)	{(1, 'John', 20), (2, 'Doe', 21)}

Q5. Stored vs. Derived Attributes

Question: Distinguish between stored and derived attributes. Give an example.

- **Stored Attribute:**
 - **Definition:** An attribute that is physically saved in the database tables.
 - **Example:** DateOfBirth in a Student table. The actual date is written to disk.
- **Derived Attribute:**
 - **Definition:** An attribute that is not permanently stored but is calculated from other attributes when needed.
 - **Example:** Age . It is calculated dynamically using the formula: $\text{CurrentDate} - \text{DateOfBirth}$.

Q6. Shared vs. Exclusive Locks

Question: Differentiate between shared and exclusive locks.

Shared Lock (S)	Exclusive Lock (X)
Purpose: Used for Read operations.	Purpose: Used for Write (Update/Delete) operations.

Concurrency: Multiple transactions can hold a Shared lock on the same data item simultaneously.

Concurrency: Only one transaction can hold an Exclusive lock on a data item at a time.

Restriction: Other transactions can read, but cannot write.

Restriction: No other transaction can read or write to the locked item.

Q7. Elasticsearch Schema

Question: Does Elasticsearch have a schema? Give reason.

Answer: Yes, but it is **schema-flexible**.

- **Reason:** Elasticsearch uses "Mappings" to define fields and their data types. However, it supports **Dynamic Mapping**. If you index a document with a new field that hasn't been defined yet, Elasticsearch automatically detects the data type (e.g., integer, string, boolean) and adds it to the mapping.
- **Summary:** It is often called "schema-less" because you don't *have* to define it upfront, but internally, a schema exists.

Part C: Transactions & Scheduling

Q8. Serial vs. Non-Serial Schedule

Question: Give an example for serial schedule and non-serial schedule.

1. Serial Schedule: Transactions satisfy all operations one after the other. No interleaving.

T1	T2

Read(A)	
Write(A)	
Commit	
	Read(A)
	Write(A)
	Commit

2. Non-Serial Schedule: Operations from different transactions are interleaved.

T1	T2

Read(A)	
	Read(A)
Write(A)	
	Write(A) <-- Potential Conflict here
Commit	
	Commit

Q9. ACID Properties

Question: List out the desirable properties of transactions.

The desirable properties are known as **ACID**:

1. **Atomicity:** The "All or Nothing" rule. A transaction must be treated as a single unit of work; either all of its operations are executed, or none are.
2. **Consistency:** The database must remain in a valid state before and after the transaction (satisfying all integrity constraints).
3. **Isolation:** Multiple transactions occurring at the same time must not affect each other. Intermediate states of T1 should not be visible to T2.
4. **Durability:** Once a transaction is committed, the changes are permanent and survive system failures (saved to non-volatile storage).

Part D: Descriptive Theory Questions (Long Answer)

Q2.a. Discuss the characteristics of the database approach. (6 marks)

The database approach differs significantly from traditional file processing. Its key characteristics include:

1. **Self-Describing Nature of a Database System:**
 - Unlike file systems, a database system contains not only the data but also the complete definition of the data structure and constraints (meta-data).
 - This definition is stored in the DBMS catalog.
2. **Insulation between Programs and Data (Program-Data Independence):**
 - In file processing, the structure of data files is embedded in the application programs. In DBMS, the structure is stored in the catalog.
 - Changes to the structure of the data (e.g., adding a new field) do not necessarily require changing the access programs.
3. **Data Abstraction:**
 - A data model is used to hide storage details and present the user with a conceptual view of the database. Users don't need to know how data is physically stored (e.g., in a B-Tree or Hash file).
4. **Support of Multiple Views of the Data:**
 - Different users may need different perspectives of the database.
 - *Example:* A student might see their grades, while an accountant sees only fee payment details. The DBMS supports defining these virtual views.
5. **Sharing of Data and Multi-user Transaction Processing:**
 - The DBMS allows multiple users to access the database at the same time.

- Concurrency control software ensures that simultaneous updates do not result in erroneous data.

6. Control of Data Redundancy:

- The database approach minimizes redundancy (duplicate data) by integrating files, which saves storage space and improves data consistency.

Q2.b. Explain the example structural constraints of relationship type. (6 marks)

Structural constraints on relationships are usually of two types: **Cardinality Ratio** and **Participation Constraint**.

1. Cardinality Ratio: Specifies the maximum number of relationship instances that an entity can participate in.

- **1:1 (One-to-One):** An employee manages exactly one department.
- **1:N (One-to-Many):** A department employs many employees, but an employee works for only one department.
- **M:N (Many-to-Many):** An employee works on many projects, and a project has many employees.

2. Participation Constraint (Existence Dependency): Specifies whether the existence of an entity depends on its being related to another entity via the relationship type.

- **Total Participation (Mandatory):**
 - Every entity in the entity set **must** participate in the relationship.
 - *Representation:* Double line in ER diagrams.
 - *Example:* Every `Loan` must belong to a `Customer`.
- **Partial Participation (Optional):**
 - Some entities in the entity set may **not** participate in the relationship.
 - *Representation:* Single line in ER diagrams.
 - *Example:* Not every `Employee` manages a `Department` (only some do).

Diagrammatic Representation (Min, Max Notation): Constraints can be represented as (min, max) next to the entity.

- min = 0 : Partial participation.
- min > 0 : Total participation.

Q2.c. Differentiate between physical data independence and logical data independence. Give an example. (4 marks)

Data independence is the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

Feature

Logical Data Independence

Physical Data Independence

Definition	The capacity to change the conceptual schema without changing the external schemas or application programs.	The capacity to change the internal schema without changing the conceptual schema.
Focus	Deals with the logical structure/definition of data.	Deals with the physical storage/organization of data.
Example	Adding a new attribute (e.g., <code>DateOfBirth</code>) to the <code>Student</code> entity should not break existing programs that print the student's Grade Report.	Creating a new index (e.g., B-Tree) on the <code>Student_ID</code> field to speed up search should not require any changes to the logical definition of <code>Student</code> .

Part E: Relational Algebra & Query Processing

Q4.a. Explain DIVISION operation of relational algebra with an Example.

Definition: The Division operation ($R \div S$) is used for queries involving the concept of "universal quantification" (e.g., "for all", "every"). It returns the set of values from relation R that are associated with **every** value in relation S .

General Form: If $R(A, B)$ and $S(B)$ are relations, then $R \div S$ returns all values of A such that for every value of B in S , the pair (A, B) exists in R .

Example: Query: Find students who have taken ALL courses listed in relation S.

Relation R (Student_Course)

Student	Course
A	DBMS
A	OS
B	DBMS
B	OS
B	Networks
C	DBMS

Relation S (Required_Courses)

Course
DBMS
OS

Result ($R \div S$):

Student

A

B

• **Reasoning:**

- Student **A** has taken both DBMS and OS. (Included)
- Student **B** has taken DBMS, OS, and Networks. (Included)
- Student **C** has taken only DBMS (Missing OS). (Excluded)

Q4.b. Relational Algebra Queries

Schema:

- STUDENT(SNO, SNAME, DEPT)
- COURSE(CNO, CNAME, DEPT)
- ENROLL(CNO, SNO, GRADE)
- PREREQ(CNO, PNO)

(i) Find names of all the students enrolled in course name (CNAME) = 'CSE562'. We first select the specific course, join with Enroll to find students, and then join with Student to get names.

$$\pi_{SNAME}(STUDENT \bowtie (\sigma_{CNAME='CSE562'}(COURSE) \bowtie ENROLL))$$

(ii) Find names of all the students who took all the courses offered by CSE department. This requires the **Division** operation.

1. **Find all CSE courses:**

$$CSE_COURSES \leftarrow \pi_{CNO}(\sigma_{DEPT='CSE'}(COURSE))$$

2. **Get all student enrollments (SNO, CNO):**

$$STUDENT_ENROLLS \leftarrow \pi_{SNO,CNO}(ENROLL)$$

3. **Divide to find students who took all CSE courses:**

$$ALL_CSE_STUDENTS \leftarrow STUDENT_ENROLLS \div CSE_COURSES$$

4. **Get Names:**

$$RESULT \leftarrow \pi_{SNAME}(STUDENT \bowtie ALL_CSE_STUDENTS)$$

(iii) For every course, list the course together with the average grade in that course. This requires the **Aggregate** function ($\mathfrak{S}$ or γ).

$$\pi_{CNO,CNAME,AVG_GRADE}(COURSE \bowtie (CNO \mathfrak{S}_{AVG(GRADE)}(ENROLL)))$$

(iv) List all the students who never got a grade above 3.0. We use the **Set Difference** approach: (All Students) - (Students who got > 3.0).

1. Find students with at least one grade > 3.0:

$$HIGH_SCORERS \leftarrow \pi_{SNO}(\sigma_{GRADE > 3.0}(ENROLL))$$

2. Find all students:

$$ALL_STUDENTS \leftarrow \pi_{SNO}(STUDENT)$$

3. Subtract high scorers from all students:

$$LOW_SCORERS \leftarrow ALL_STUDENTS - HIGH_SCORERS$$

4. Get Names:

$$RESULT \leftarrow \pi_{SNAME}(STUDENT \bowtie LOW_SCORERS)$$

(v) Find names of all the courses in which more than 10 students have enrolled. This requires grouping and a condition (similar to SQL `HAVING`).

1. Count students per course:

$$COURSE_COUNTS \leftarrow_{CNO} \mathfrak{S}_{COUNT(SNO)}(ENROLL)$$

2. Select courses with count > 10:

$$POPULAR_COURSES \leftarrow \sigma_{COUNT > 10}(COURSE_COUNTS)$$

NHL Database System: ER Diagram & Analysis

Question 3.a Solution

1. Assumptions and Constraints

Based on the requirements provided, we make the following design assumptions to ensure the ER diagram is robust:

1. **Entity Keys:**
- Team:** Name is unique and acts as the Primary Key (PK).
 - Player:** While names can be duplicated, for this model we assume PlayerID (surrogate key) or Name is the PK.
 - Game:** Identified by a unique GameID or the combination of {Date, HostTeam, GuestTeam} .
 - Injury Record:** Treated as a **Weak Entity** dependent on the Player (since it has no ID of its own, just a "set" of records per player).
2. **Relationships:**
- Team-Player (Roster):** A 1:N relationship. A Team has many Players; a Player belongs to exactly one Team.
 - Team-Captain:** A 1:1 relationship. A Team has exactly one Captain, and that Captain is a specific Player.
 - Game-Team:** Two relationships exist here with specific **roles**:
 - Host Team (1 Game has 1 Host; 1 Team hosts N Games).
 - Guest Team (1 Game has 1 Guest; 1 Team guests in N Games).
3. **Attributes:**
- The Score attribute in **Game** is derived or simple.
 - Injury Records are modeled as a weak entity to allow storing details (like injury type, date) rather than just a multi-valued attribute string.

2. Cardinality and Role Indicators

Relationship	Entity A	Entity B	Cardinality	Role Indicator	Description
Plays_For	Team	Player	1:N	-	One team has many players; a player has 1 team.
Leads	Team	Player	1:1	Captain	One team has one captain; one player is captain of 1 team.

Hosts	Team	Game	1:N	Host	A team hosts many games; a game has one host.
Visits	Team	Game	1:N	Guest	A team visits for many games; a game has one guest.
Suffers	Player	Injury	1:N	-	A player has many injuries; an injury belongs to 1 player.

3. ER Diagram

```
erDiagram
    TEAM {
        string Name PK
        string City
        string Coach
    }

    PLAYER {
        string Name
        string Position
        string SkillLevel
        int PlayerID PK
    }

    GAME {
        int GameID PK
        date Date
        string Score
    }

    INJURY_RECORD {
        string Description
        date Date
    }

    %% Relationship: Roster
    TEAM ||--}| PLAYER : "employs/roster"

    %% Relationship: Captaincy
    TEAM ||--|| PLAYER : "is captained by"

    %% Relationship: Game Hosting (Roles)
    TEAM ||--o{ GAME : "hosts"
    TEAM ||--o{ GAME : "is guest in"

    %% Relationship: Injuries (Weak Entity)
    PLAYER ||--o{ INJURY_RECORD : "has"
```

4. Design Explanations

1. The "Captain" Relationship: You will notice two lines connecting TEAM and PLAYER .
- The first line (||--|{) represents the standard roster: A team has many players.

- The second line (| | - - | |) represents the captaincy: A specific player acts as the captain for the team. This satisfies the requirement "A team captain is also a player."
2. **The "Game" Relationship:** A game is an interaction between two teams. We model this as two separate one-to-many relationships (Host and Guest) rather than a ternary relationship, as it clearly defines the role of each team in that specific match.
 3. **Injury Records:** Since the prompt asks for a "set of injury records," modeling this as a separate box (Weak Entity) connected to Player allows us to store multiple injuries for a single player without violating First Normal Form (1NF).

Question 3.b Solution

Constraints Violations & Recovery Actions

Question: Explain how relational model constraints may be violated by insert/delete operations and describe actions to take. (8 marks)

1. Violations by Operation

A. INSERT Operation Violations When trying to insert a new tuple (row) into a relation, four types of constraints can be violated:

1. **Domain Constraint Violation:**
 - **Cause:** Inserting a value that is not of the correct data type or range defined for that column.
 - *Example:* Inserting the string "Twenty" into an `Age` (integer) column.
2. **Key Constraint Violation:**
 - **Cause:** Inserting a tuple with a Primary Key value that already exists in the table.
 - *Example:* Inserting a new Student with `ID=101` when `101` already exists.
3. **Entity Integrity Constraint Violation:**
 - **Cause:** Inserting a tuple where the Primary Key attribute is `NULL`.
 - *Rule:* Primary keys cannot be null because they uniquely identify tuples.
4. **Referential Integrity Constraint Violation:**
 - **Cause:** Inserting a tuple where the Foreign Key value refers to a non-existent value in the parent table.
 - *Example:* Assigning a Student to `Dept_ID=5` when Department 5 does not exist.

B. DELETE Operation Violations Deleting a tuple can violate only one major constraint:

1. **Referential Integrity Constraint Violation:**
 - **Cause:** Trying to delete a tuple from a **Parent Table** (referenced table) while there are still tuples in the **Child Table** (referencing table) pointing to it.
 - *Example:* Deleting a `Department` that still has `Employees` assigned to it.

2. Actions Taken by DBMS upon Violation

When an update operation (Insert, Delete, or Update) violates a constraint, the DBMS can handle it in one of the following ways (usually defined in the Schema via DDL):

1. Restrict / Reject (Default Action)

- **Description:** The DBMS simply refuses to execute the operation and returns an error message to the user.
- **Usage:** Used for Domain, Key, and Entity Integrity violations always. Also the default for Referential Integrity.
- *Example:* "Error: Cannot delete Department 5 because child records exist."

2. Cascade (Propagation)

- **Description:** The operation performed on the parent tuple is automatically applied to the referencing child tuples.
- **Usage:** Common for **Delete** operations involving Referential Integrity.
- *Example (ON DELETE CASCADE):* If Department 5 is deleted, all Employees belonging to Department 5 are **also automatically deleted**.

3. Set Null

- **Description:** The referencing attribute in the child tuple is set to NULL .
- **Usage:** Valid only if the Foreign Key column allows NULL values.
- *Example (ON DELETE SET NULL):* If Department 5 is deleted, the Dept_ID for all its employees is set to NULL (indicating they are currently unassigned), but the employee records remain.

4. Set Default

- **Description:** The referencing attribute is set to a predefined default value.
- **Usage:** Valid only if a default value is defined for the column.
- *Example (ON DELETE SET DEFAULT):* If a Team is deleted, the players are reassigned to a default "Free Agents" team ID.

Solved DBMS Exam: Question 5 & 6

Topic: Normalization, SQL, Anomalies, and Integrity Constraints

5.a. Normalization and 2NF Decomposition (10 Marks)

Problem Statement: Consider the relation scheme $R(A, B, C, D, E, F, G, H, I, J, K, L)$ with the set of functional dependencies:

$$F = \{\{A, B\} \rightarrow \{C\}, \{B\} \rightarrow \{F\}, \{F\} \rightarrow \{G, H\}, \{D\} \rightarrow \{I, J\}, \{B\} \rightarrow \{K, L\}\}$$

Candidate Keys: $\{A, B, D\}$ and $\{K\}$.

(i) Find the FDs which do not satisfy the 2NF test

Definition of 2NF: A relation is in Second Normal Form (2NF) if it is in 1NF and **no non-prime attribute is partially dependent on any candidate key**.

- **Prime Attributes:** Attributes that are part of a candidate key. Here: $\{A, B, D, K\}$.
- **Non-Prime Attributes:** $\{C, E, F, G, H, I, J, L\}$.
- **Partial Dependency:** A dependency $X \rightarrow Y$ where X is a *proper subset* of a candidate key.

Analysis of FDs against Key $\{A, B, D\}$:

1. $AB \rightarrow C$:
 - LHS $\{A, B\}$ is a proper subset of key $\{A, B, D\}$.
 - RHS C is non-prime.
 - **Result:** Violates 2NF (Partial Dependency).
2. $B \rightarrow F$:
 - LHS $\{B\}$ is a proper subset of key $\{A, B, D\}$.
 - RHS F is non-prime.
 - **Result:** Violates 2NF (Partial Dependency).
3. $F \rightarrow G, H$:
 - LHS $\{F\}$ is not a subset of the key.
 - **Result:** Does not violate 2NF (This is a Transitive Dependency, relevant for 3NF).
4. $D \rightarrow I, J$:
 - LHS $\{D\}$ is a proper subset of key $\{A, B, D\}$.
 - RHS I, J are non-prime.
 - **Result:** Violates 2NF (Partial Dependency).
5. $B \rightarrow K, L$:

- Splitting into $B \rightarrow K$ and $B \rightarrow L$.
- $B \rightarrow L$: LHS $\{B\}$ is proper subset; L is non-prime. **Violates 2NF.**
- $B \rightarrow K$: K is a prime attribute (part of candidate key $\{K\}$). Technically allowed in 2NF, but usually decomposed with B 's other dependencies.

Answer: The FDs violating 2NF are:

- $AB \rightarrow C$
- $B \rightarrow F$
- $D \rightarrow I, J$
- $B \rightarrow L$

(ii) Decompose R into 2NF relations

To normalize to 2NF, we decompose the relation to remove the partial dependencies. We create separate relations for the dependent attributes and their determinants.

Decomposition Steps:

1. **Relation R_1 (from $AB \rightarrow C$):**
 - Attributes: $\{A, B, C\}$
 - Key: $\{A, B\}$
2. **Relation R_2 (from $B \rightarrow F, K, L$):**
 - Attributes: $\{B, F, K, L\}$
 - Key: $\{B\}$
 - *Note: We group F, K , and L here as they all depend on B .*
3. **Relation R_3 (from $D \rightarrow I, J$):**
 - Attributes: $\{D, I, J\}$
 - Key: $\{D\}$
4. **Relation R_4 (from $F \rightarrow G, H$):**
 - Attributes: $\{F, G, H\}$
 - Key: $\{F\}$
 - *Note: Although $F \rightarrow G, H$ is a transitive dependency (3NF issue), it is best practice to separate it during decomposition to ensure a clean design.*
5. **Relation R_5 (The Residual/Main Relation):**
 - Attributes: Remaining attributes + Key.
 - We must keep the candidate key $\{A, B, D\}$ and any attributes not removed (E).
 - Attributes: $\{A, B, D, E\}$
 - Key: $\{A, B, D\}$

Final 2NF Relations:

- $R_1(A, B, C)$
- $R_2(B, F, K, L)$
- $R_3(D, I, J)$
- $R_4(F, G, H)$
- $R_5(A, B, D, E)$

5.b. SQL Queries

Schema:

- Supplier(sid: int, sname: string, city: string)
- Parts(pid: int, pname: string, color: string)
- Catalog(sid: int, pid: int, cost: real)

(i) Find pid for parts supplied by supplier name 'Ramesh'.

```
SELECT DISTINCT C.pid
FROM Catalog C
JOIN Supplier S ON C.sid = S.sid
WHERE S.sname = 'Ramesh';
```

(ii) Find the number of suppliers who supply red parts.

```
SELECT COUNT(DISTINCT C.sid)
FROM Catalog C
JOIN Parts P ON C.pid = P.pid
WHERE P.color = 'Red';
```

Note: DISTINCT is crucial here because a supplier might supply multiple different red parts.

(iii) Find the number of parts supplied by each supplier.

```
SELECT S.sid, S.sname, COUNT(C.pid) AS total_parts
FROM Supplier S
LEFT JOIN Catalog C ON S.sid = C.sid
GROUP BY S.sid, S.sname;
```

Note: LEFT JOIN is used to include suppliers who supply 0 parts.

5.c. Insert and Delete Anomalies

Anomalies are problems that occur in poorly structured (unnormalized) databases when performing update operations.

1. Insert Anomaly

Definition: An insert anomaly occurs when certain attributes cannot be inserted into the database without the presence of other unrelated attributes.

Example: Consider the unnormalized relation from Question 5.a: $R(A, B, D, I, J, \dots)$ where Key is $\{A, B, D\}$ and $D \rightarrow I, J$.

- Suppose D represents a "Department" and I, J are "DeptName" and "Location".
- If we want to create a new Department D_{new} , we cannot insert it into relation R until we also have values for A (Student?) and B (Course?).
- We are forced to either wait or insert NULL values for the primary key (which is not allowed).
- **Solution:** By decomposing into $R_3(D, I, J)$, we can insert a new Department independently.

2. Delete Anomaly

Definition: A delete anomaly occurs when deleting a record results in the unintended loss of other valid data.

Example: Consider the same relation R .

- Suppose we have only one tuple where Department D_1 is used (e.g., only one student A_1 is enrolled in a course B_1 associated with Department D_1).
- If we delete that specific tuple (perhaps the student dropped the course), we also delete the values associated with D_1 (attributes I and J).
- The information about Department D_1 is lost entirely from the database, which is unintended.
- **Solution:** By decomposing into $R_3(D, I, J)$, deleting a tuple in the main relation R_5 does not affect the existence of the department record in R_3 .

6.a. Normal Forms (2NF & 3NF) Explained

Question: With an Example for each, explain 2NF, 3NF based on primary keys.

1. Second Normal Form (2NF)

Definition: A relation is in 2NF if:

1. It is already in **1NF** (First Normal Form).
 2. **No Partial Dependency** exists. This means no non-prime attribute should depend on a proper subset of a composite candidate key.
- **Partial Dependency:** If Key is (A, B) and $A \rightarrow C$ (where C is non-prime), then C is partially dependent on the key.

Example (Violation & Fix):

- **Relation:** GRADES(Student_ID, Course_ID, Student_Name, Grade)
- **Key:** {Student_ID, Course_ID}

- **Dependency:** $\text{Student_ID} \rightarrow \text{Student_Name}$
- **Violation:** Student_Name depends only on Student_ID (part of the key), not the full key. This is a partial dependency.
- **Normalization:** Split into:
 1. $\text{STUDENT}(\text{Student_ID}, \text{Student_Name})$
 2. $\text{GRADES}(\text{Student_ID}, \text{Course_ID}, \text{Grade})$

2. Third Normal Form (3NF)

Definition: A relation is in 3NF if:

1. It is already in **2NF**.
 2. **No Transitive Dependency** exists. No non-prime attribute should depend on another non-prime attribute.
- **Transitive Dependency:** If $A \rightarrow B$ (Key determines B) and $B \rightarrow C$ (Non-key determines Non-key), then $A \rightarrow C$ is transitive.

Example (Violation & Fix):

- **Relation:** $\text{EMPLOYEE}(\text{Emp_ID}, \text{Dept_ID}, \text{Dept_Name})$
- **Key:** $\{\text{Emp_ID}\}$
- **Dependency:** $\text{Emp_ID} \rightarrow \text{Dept_ID}$ and $\text{Dept_ID} \rightarrow \text{Dept_Name}$.
- **Violation:** Dept_Name is determined by Dept_ID , which is not a key. Dept_Name transitively depends on Emp_ID .
- **Normalization:** Split into:
 1. $\text{EMPLOYEE}(\text{Emp_ID}, \text{Dept_ID})$
 2. $\text{DEPARTMENT}(\text{Dept_ID}, \text{Dept_Name})$

6.b. Lossless Join Property Test

Question:

- **Relation:** $R = \{A, B, C, D, E, F, G, H, I, J\}$
- **FDs:**

$$F = \{\{A, B\} \rightarrow \{C\}, \{A\} \rightarrow \{D, E\}, \{B\} \rightarrow \{F\}, \{F\} \rightarrow \{G, H\}, \{D\} \rightarrow \{I, J\}\}$$
- **Decomposition:** $D = \{R_1, R_2, R_3, R_4, R_5\}$ where:
 - $R_1 = \{A, B, C\}$
 - $R_2 = \{A, D, E\}$
 - $R_3 = \{B, F\}$
 - $R_4 = \{F, G, H\}$
 - $R_5 = \{D, I, J\}$

Analysis: To verify if the decomposition is **Lossless**, we check if the intersection of the relations is a key (determinant) for at least one of them at every join step.

Step 1: Join $R_1(A, B, C)$ and $R_2(A, D, E)$

- **Intersection:** $R_1 \cap R_2 = \{A\}$
- **Check Determinant:** In F , we have $A \rightarrow \{D, E\}$. Since A determines the attributes of R_2 (minus A), the join is lossless.
- **Result:** $S_{12} = \{A, B, C, D, E\}$

Step 2: Join S_{12} and $R_3(B, F)$

- **Intersection:** $S_{12} \cap R_3 = \{B\}$
- **Check Determinant:** In F , we have $B \rightarrow \{F\}$. Since B determines R_3 , the join is lossless.
- **Result:** $S_{123} = \{A, B, C, D, E, F\}$

Step 3: Join S_{123} and $R_4(F, G, H)$

- **Intersection:** $S_{123} \cap R_4 = \{F\}$
- **Check Determinant:** In F , we have $F \rightarrow \{G, H\}$. Since F determines R_4 , the join is lossless.
- **Result:** $S_{1234} = \{A, B, C, D, E, F, G, H\}$

Step 4: Join S_{1234} and $R_5(D, I, J)$

- **Intersection:** $S_{1234} \cap R_5 = \{D\}$
- **Check Determinant:** In F , we have $D \rightarrow \{I, J\}$. Since D determines R_5 , the join is lossless.
- **Result:** $S_{Final} = \{A, B, C, D, E, F, G, H, I, J\}$ (Original Relation R)

Conclusion: Since we could reconstruct the original relation R without creating spurious tuples (every step satisfied the lossless join condition), the decomposition is **Lossless**.

6.c. SQL Implementation of Referential Integrity (4 marks)

Explanation: SQL implements Referential Integrity using the **Foreign Key (FK)** constraint.

- A Foreign Key is a column (or set of columns) in a table that refers to the **Primary Key** of another table.
- It ensures that the value in the child table matches a valid value in the parent table or is NULL.

SQL Syntax & Actions: The REFERENCES clause links the tables. You can also specify actions for updates/deletes:

- ON DELETE CASCADE : Deleting a parent row deletes all matching child rows.
- ON DELETE SET NULL : Deleting a parent row sets the FK in child rows to NULL.

Example:

```
/* Parent Table */
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

/* Child Table with Referential Integrity */
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    /* Constraint Definition */
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
    ON DELETE CASCADE
);
```

In this example, if a `Department` is deleted, all its `Employees` are automatically deleted to preserve integrity.

Solved DBMS Exam: Question 7 & 8

Topic: Transactions, Concurrency Control, Serializability, and 2PL

7.a. Transaction States with State Diagram

Question: Explain the transaction states with the state diagram.

Answer: A transaction passes through several states during its execution to ensure data reliability and consistency.

Transaction States

1. **Active:** The initial state. The transaction stays in this state while it is executing read or write operations.
2. **Partially Committed:** The transaction enters this state after the final operation has been executed. It is waiting for the final check (e.g., ensuring updates can be written to disk) before committing.
3. **Failed:** The transaction enters this state if a hardware failure or logical error occurs in the Active or Partially Committed state, making normal execution impossible.
4. **Aborted:** After the transaction has failed and the database has been rolled back to its state prior to the start of the transaction. From here, the transaction can be restarted or killed.
5. **Committed:** The transaction enters this state after successful completion. All changes are permanently saved (durability).
6. **Terminated:** The final state where the transaction leaves the system.

Transaction State Diagram

```
stateDiagram-v2
    [*] --> Active : Begin Transaction
    Active --> PartiallyCommitted : End of Transaction
    Active --> Failed : Error/Crash
    PartiallyCommitted --> Committed : Commit
    PartiallyCommitted --> Failed : Error
    Failed --> Aborted : Rollback
    Aborted --> Terminated
    Committed --> Terminated
    Terminated --> [*]
```

7.b. Serializability Graph

Question: Consider the 3 transactions T1, T2, T3 and schedule S1. Draw the serializability graph for S1 and state if the schedule is serializable.

Transactions:

- $T1 : r_1(X); r_1(Z); w_1(X)$
- $T2 : r_2(Z); r_2(Y); w_2(Z); w_2(Y)$
- $T3 : r_3(X); r_3(Y); w_3(Y)$

Schedule S1:

$$r_1(X), r_2(Z), r_1(Z), r_3(X), r_3(Y), w_1(X), w_3(Y), r_2(Y), w_2(Z), w_2(Y)$$
Step 1: Identify Conflicting Operations

We look for pairs of operations on the **same data item** by **different transactions** where **at least one is a write**.

1. Item X:

- $r_3(X)$ (step 4) occurs before $w_1(X)$ (step 6).
- **Conflict:** $T3 \rightarrow T1$

2. Item Z:

- $r_1(Z)$ (step 3) occurs before $w_2(Z)$ (step 9).
- **Conflict:** $T1 \rightarrow T2$

3. Item Y:

- $w_3(Y)$ (step 7) occurs before $r_2(Y)$ (step 8).
- **Conflict:** $T3 \rightarrow T2$
- $w_3(Y)$ (step 7) occurs before $w_2(Y)$ (step 10).
- **Conflict:** $T3 \rightarrow T2$ (Redundant)

Step 2: Draw Precedence Graph

- **Nodes:** T1, T2, T3
- **Edges:**
 - $T3 \rightarrow T1$
 - $T1 \rightarrow T2$
 - $T3 \rightarrow T2$

```
graph LR
```

```
T3((T3)) --> T1((T1))
```

```
T1 --> T2((T2))
```

```
T3 --> T2
```

Step 3: Check for Cycles

- We traverse the graph: $T3 \rightarrow T1 \rightarrow T2$.
- There are **NO cycles** in the graph.

Conclusion

Since the precedence graph is acyclic, the schedule S1 is **Conflict Serializable**.

Equivalent Serial Order: $T3 \rightarrow T1 \rightarrow T2$

7.c. Two-Phase Locking (2PL) Protocol

Question: Explain with an example the 2-phase locking protocol.

Definition: The Two-Phase Locking (2PL) protocol is a concurrency control mechanism that ensures conflict serializability. It dictates that a transaction must acquire all necessary locks before it releases any lock.

The Two Phases

1. Growing Phase:

- The transaction acquires all the locks (Shared or Exclusive) it needs.
- The transaction **cannot release** any lock in this phase.

2. Shrinking Phase:

- The transaction releases its locks.
- Once the first lock is released, the transaction enters this phase and **cannot acquire** any new locks.

Lock Types

- **Shared Lock (S):** Required to Read a data item. Multiple transactions can hold S-locks on the same item.
- **Exclusive Lock (X):** Required to Write (update) a data item. Only one transaction can hold an X-lock on an item.

Example

Consider two transactions, T1 and T2.

Without 2PL (Interleaved - Potential Problem): If T1 locks A, unlocks A, then locks B, T2 might access A in between, leading to inconsistency.

With 2PL (Enforced phases):

Step	Transaction T1	Transaction T2	Phase (T1)
1	Lock_X(A)		Growing

2	Read(A)	Lock_S(A) (WAIT)	Growing
3	Write(A)		Growing
4	Lock_X(B)		Growing (Point of Max Locking)
5	Unlock(A)		Shrinking (Cannot lock anymore)
6		Lock_S(A) (Acquired)	
7	Write(B)	Read(A)	Shrinking
8	Unlock(B)		Shrinking

Explanation of Example:

1. T1 acquires locks on A and B (Growing).
2. Even though T1 finished with A at step 3, it holds the lock until it acquires the lock on B (Step 4) to ensure the 2PL rule is met.
3. T2 has to wait until T1 enters the shrinking phase and releases A.
4. This guarantees that all conflicting operations appear in a serial order.

8.a. Why Concurrency Control is Needed?

Question: Why Concurrency control is needed? Explain with an Example. (8 marks)

Answer: Concurrency control is required in a DBMS to manage the simultaneous execution of multiple transactions. Its primary goals are to preserve the consistency and correctness of the database and to ensure the property of Isolation (from ACID).

Without concurrency control, uncontrolled interleaved operations may lead to data inconsistency and anomalies.

Common Concurrency Anomalies:

1. **Lost Update:** Two transactions update the same data item, and one update overwrites the other.
2. **Dirty Read:** A transaction reads data written by another transaction that has not yet committed.
3. **Unrepeatable Read:** A transaction reads the same item twice and gets different values because another transaction modified it.

Example: The Lost Update Problem

Consider a bank account with an initial balance $X = 100$. Two transactions, T_1 (Deposit 50) and T_2 (Withdraw 30), execute concurrently without proper locking.

Schedule (Interleaved):

Time	Transaction T1	Transaction T2	Value of X (Database)
1	Read(X) (Reads 100)		100
2		Read(X) (Reads 100)	100
3	$X = X + 50$ (Local: 150)		100
4	Write(X) (Writes 150)		150
5		$X = X - 30$ (Local: 70)	150
6		Write(X) (Writes 70)	70

Analysis:

- $T1$ updates X to 150.
- $T2$ overwrites X with 70 (based on the stale value of 100 it read earlier).
- **Result:** The deposit of 50 is lost. The correct value should be $100 + 50 - 30 = 120$.
- **Conclusion:** Concurrency control prevents this by forcing $T2$ to wait until $T1$ finishes.

8.b. Serializability & Testing Algorithm

Question: Define Serializability, Conflict Serializability. With an Example explain the algorithm for Testing Conflict Serializability of a Schedule. (8 marks)

Definitions

1. Serializability: A concurrent schedule is **serializable** if its final effect on the database and the values read by transactions are equivalent to some **serial schedule** (where transactions are executed one after the other, with no interleaving).

2. Conflict Serializability: A schedule is **conflict serializable** if it can be transformed into a serial schedule by swapping non-conflicting adjacent operations.

- **Conflicting Operations:** Two operations conflict if:
 1. They belong to different transactions.
 2. They access the same data item.
 3. At least one of them is a Write operation.

Algorithm for Testing Conflict Serializability

We use the **Precedence Graph (Serialization Graph)** method.

Steps:

1. **Create Nodes:** Create a node for each transaction participating in the schedule.

2. **Add Edges:** For every pair of conflicting operations Op_i and Op_j belonging to transactions T_i and T_j respectively:
 - If Op_i occurs before Op_j , draw a directed edge $T_i \rightarrow T_j$.
3. **Check Cycles:** Check the graph for a cycle.
 - **No Cycle:** The schedule is Conflict Serializable.
 - **Cycle Exists:** The schedule is NOT Conflict Serializable.

Example

Consider the following Schedule S :

$$S : r_1(X), w_1(X), r_2(X), w_2(X)$$

Step 1: Identify Conflicts

- **Conflict 1:** $w_1(X)$ happens before $r_2(X)$.
 - Type: Write-Read Conflict.
 - Edge: $T_1 \rightarrow T_2$.
- **Conflict 2:** $w_1(X)$ happens before $w_2(X)$.
 - Type: Write-Write Conflict.
 - Edge: $T_1 \rightarrow T_2$.

Step 2: Draw Precedence Graph

```
graph LR
  T1((T1)) --> T2((T2))
```

Step 3: Check Cycles

- The graph contains edges $T_1 \rightarrow T_2$.
- There is **NO cycle** in the graph.

Conclusion: Since the precedence graph is acyclic, the schedule is **Conflict Serializable**. The

Solved DBMS Exam: Question 9 & 10

Topic: NoSQL, Elasticsearch, Hadoop (Big Data), and Data Types

9.a. Discuss Tokenizer and Index in Elasticsearch

1. Tokenizer

A **Tokenizer** is a core component of the text analysis process in Elasticsearch. It receives a stream of characters, breaks it up into individual terms (tokens), and outputs a stream of tokens.

- **Role:**
 - Converts raw text into a format suitable for searching.
 - It is used during both **Indexing** (when data is saved) and **Searching** (when data is queried).
 - It handles whitespace, punctuation, and special characters based on specific rules.
- **Example:**
 - **Input Text:** "Big Data Systems"
 - **Standard Tokenizer Output:** [big, data, systems] (Lowercased and split by whitespace).
- **Common Types:**
 - **Standard Tokenizer:** Splits on word boundaries, removes punctuation (Default).
 - **Whitespace Tokenizer:** Splits text whenever it encounters a whitespace.
 - **Keyword Tokenizer:** Treats the entire input string as a single token (useful for IDs or emails).
 - **Edge N-gram:** Used for "search as you type" (autocomplete) functionality.

2. Index

An **Index** in Elasticsearch is a logical namespace that maps to one or more primary shards and can have zero or more replica shards. It is where related documents are stored.

- **Analogy:** If Elasticsearch is a Relational Database, then an **Index** is roughly equivalent to a **Database** (or a Table in some loose comparisons).
- **Characteristics:**
 - **Schema-Free (Flexible):** While it uses mappings to define types, fields can be added dynamically.
 - **JSON Documents:** Stores data as serialized JSON documents.
 - **Inverted Index:** Internally, it uses an inverted index data structure (like the index at the back of a book) to allow for very fast full-text searches.

Relationship

When a document is added to an **Index**, the text fields are passed through an analyzer (which includes a **Tokenizer**). The resulting tokens are stored in the **Inverted Index** to enable efficient searching.

9.b. Hadoop Distributed File System (HDFS) Architecture

Definition: HDFS is a distributed file system designed to run on commodity hardware. It is highly fault-tolerant and is designed to be deployed on low-cost hardware. It provides high throughput access to application data and is suitable for applications that have large data sets.

Architecture Components

1. NameNode (Master):

- The centerpiece of the HDFS file system.
- **Function:** Manages the file system namespace (metadata), including the mapping of file blocks to DataNodes. It regulates access to files by clients.
- **Storage:** Stores metadata in memory (RAM) for fast access.
- **SPOF:** It is a Single Point of Failure (unless High Availability is configured).

2. DataNode (Slave):

- The worker nodes in the file system.
- **Function:** Responsible for serving read and write requests from the file system clients. They also perform block creation, deletion, and replication upon instruction from the NameNode.
- **Heartbeat:** Sends periodic heartbeats to the NameNode to report health.

3. Secondary NameNode:

- **Not a backup:** Contrary to its name, it is not a hot backup for the NameNode.
- **Function:** It periodically merges the namespace image with the edit log to prevent the edit log from becoming too large (Checkpointing).

4. Blocks:

- HDFS splits files into large blocks (default 128MB or 256MB) and distributes them across nodes.
- **Replication:** Each block is replicated (default 3 copies) across different DataNodes to ensure fault tolerance.

Architecture Sketch

```
graph TD
    Client[Client Application]
    subgraph Master_Node [Master Node]
        NN[NameNode <br/> (Metadata & Block Map)]
    end
```

```

SNN[Secondary NameNode<br/>(Checkpoints)]
end

subgraph Slave Nodes
DN1[DataNode 1<br/>Block A, B]
DN2[DataNode 2<br/>Block A, C]
DN3[DataNode 3<br/>Block B, C]
end

%% Interactions
Client -- 1. Metadata Ops (Open/Close/Rename) --> NN
Client -- 2. Read/Write Blocks --> DN1
Client -- 2. Read/Write Blocks --> DN2

NN -. Checkpoint Ops .- SNN

DN1 -- Heartbeat & Block Report --> NN
DN2 -- Heartbeat & Block Report --> NN
DN3 -- Heartbeat & Block Report --> NN

style NN fill:#f9f,stroke:#333,stroke-width:2px
style DN1 fill:#bbf,stroke:#333,stroke-width:2px
style DN2 fill:#bbf,stroke:#333,stroke-width:2px
style DN3 fill:#bbf,stroke:#333,stroke-width:2px

```

9.c. Key Features of MongoDB NoSQL Database

MongoDB is a document-oriented database designed for ease of development and scaling.

1. Document-Oriented Storage:

- Stores data in **BSON** (Binary JSON) format.
- Data is stored in documents, which are grouped into collections.
- Documents align well with objects in programming code.

2. Schema Flexible (Dynamic Schema):

- MongoDB does not require a predefined schema.
- Documents in the same collection can have different fields and structures.
- Fields can be added or removed on the fly.

3. High Scalability (Sharding):

- Supports horizontal scaling through **Sharding**.
- Data is distributed across multiple machines to handle large datasets and high throughput operations.

4. High Availability (Replica Sets):

- Uses **Replica Sets** to provide automatic failover and data redundancy.
- If a primary node fails, a secondary node is automatically elected as the new primary.

5. Rich Query Language:

- Supports ad-hoc queries, filtering, and sorting.

- Supports regular expressions and searching inside nested arrays/objects.

6. Indexing:

- Supports secondary indexes on any field in the document.
- Includes specialized indexes like Geospatial (for maps) and Text Search.

7. Aggregation Framework:

- Provides a powerful framework (pipeline approach) for data processing and transformation (similar to SQL `GROUP BY` but more flexible).

10.a. Discuss MapReduce Programming Model

The **MapReduce Programming Model** is a distributed data processing framework used in Hadoop to process large datasets in parallel across multiple nodes (commodity hardware). It simplifies parallel processing by abstracting the complexity of synchronization and fault tolerance.

Working Phases

It works by dividing the computation into two main phases:

1. Map Phase:

- **Input:** The input dataset is split into independent chunks.
- **Process:** The **Mapper** function processes each input record and generates intermediate (key, value) pairs.
- **Example (Word Count):**
 - Input: "Big Data Big"
 - Map Output: (Big, 1), (Data, 1), (Big, 1)

2. Shuffle and Sort (Intermediate Phase):

- The framework groups the intermediate key-value pairs by key.
- All values associated with the same key are sent to the same **Reducer**.
- **Output:** Big -> [1, 1] , Data -> [1]

3. Reduce Phase:

- **Process:** The **Reducer** function takes the grouped data and aggregates the values for each key to produce the final output.
- **Example:**
 - Sums the list [1, 1] for "Big".
 - **Final Output:** (Big, 2), (Data, 1)

Characteristics

- **Parallel Processing:** Tasks are executed simultaneously on multiple nodes.

- **Fault Tolerance:** If a node fails, the master node re-schedules the task on another node.
- **Data Locality:** Moves computation to the data (processing happens where data is stored) rather than moving data to computation.

10.b. Types of Data with Examples

Big Data is generally classified into three types based on its structure and organization.

i) Structured Data

Definition: Data that is organized in a fixed format or strict schema, typically stored in rows and columns. It is highly organized and easy to search.

- **Characteristics:** Predefined schema, easy to query using SQL, stored in RDBMS.
- **Example:** A Relational Database Table. | ID | Name | Age | | :--- | :--- | :--- | | 1 | Ravi | 21 |

ii) Semi-Structured Data

Definition: Data that does not follow a rigid schema (like tables) but contains tags, markers, or separators to organize the data hierarchy.

- **Characteristics:** Flexible structure, "Self-describing" (contains metadata), stored in NoSQL databases or XML files.
- **Example (JSON):**

```
{
  "name": "Anita",
  "dept": "CSE",
  "marks": [85, 90]
}
```

iii) Unstructured Data

Definition: Data that has no predefined format or organization. It cannot be easily stored in a traditional relational database.

- **Characteristics:** Difficult to analyze using traditional tools, makes up the largest portion of Big Data today.
- **Examples:** Images, Videos, Audio files, PDF documents, Social media posts, Log files.

Comparison Table

Feature	Structured	Semi-Structured	Unstructured
Schema	Fixed / Rigid	Flexible / Dynamic	None
Format	Tables (Rows/Cols)	JSON, XML, CSV	Audio, Video, Text

Storage	RDBMS (MySQL, Oracle)	NoSQL (MongoDB), XML Files	Data Lakes, Blob Storage
Ease of	Very Easy (SQL)	Moderate (Parsing required)	Difficult (Requires AI/ML)